

L2 - 12.4. Threat Hunting with Zui

Task 1 - Introduction

Hunting vs Alert Response: Deductive alert verification versus inductive hypothesis-driven threat hunting.

Three Hunting Postures: Intel, anomaly, and hypothesis-driven hunt postures.



Zui provides a unified, interactive query platform that bridges logs and packet captures for rapid exploratory threat hunting.

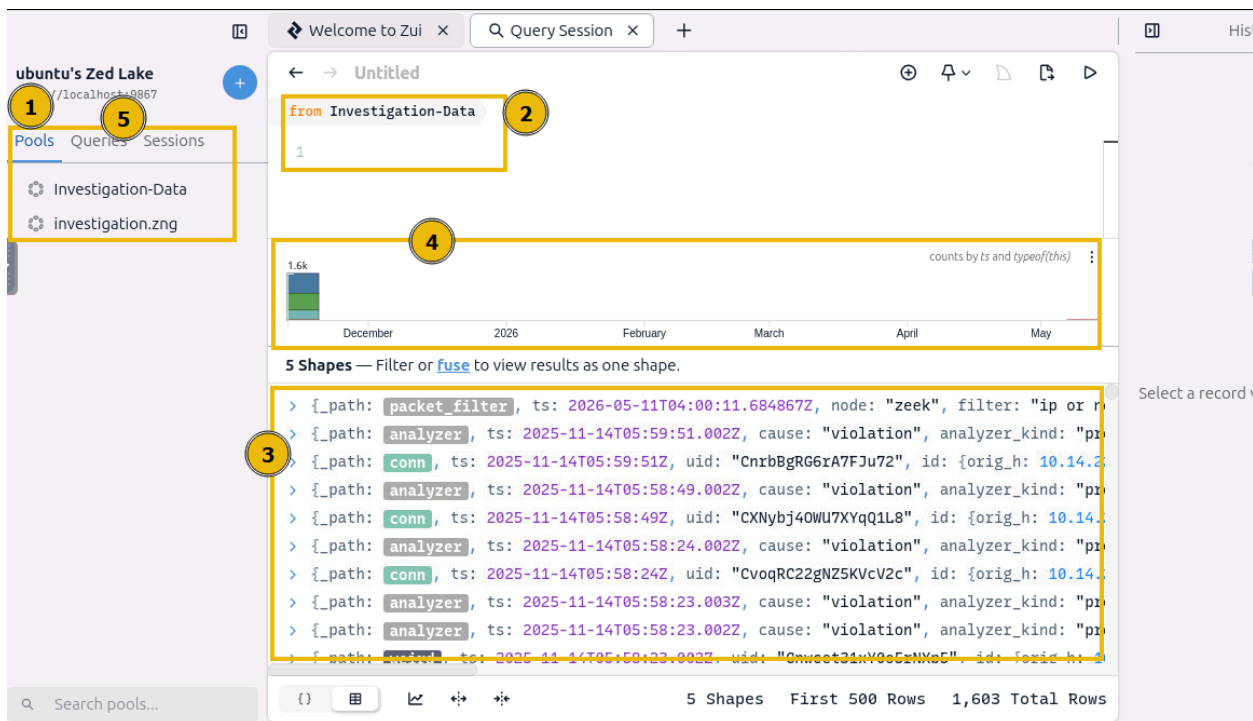
What This Room Covers

- Zui orientation, the Zed lake model, and the visual query interface
- The Zed query language for filtering, grouping, aggregating, and pivoting
- Hunting C2 beaconing, exfiltration, and lateral movement
- Building a saved query library that turns ad-hoc hunts into repeatable team workflows

Task 2 - Zui: An Overview

Zui is an interactive desktop application built on the **Zed data lake**. It bundles Zeek, the Zed command-line interface tools (`zq` and `zed`), and a powerful query engine. When you load a PCAP, Zui automatically runs Zeek in the background to generate connection, DNS, HTTP, SSL, and file logs, converts them to Zed's columnar ZNG format, and imports them into a queryable database segment called a **pool**.

The Zui Interface



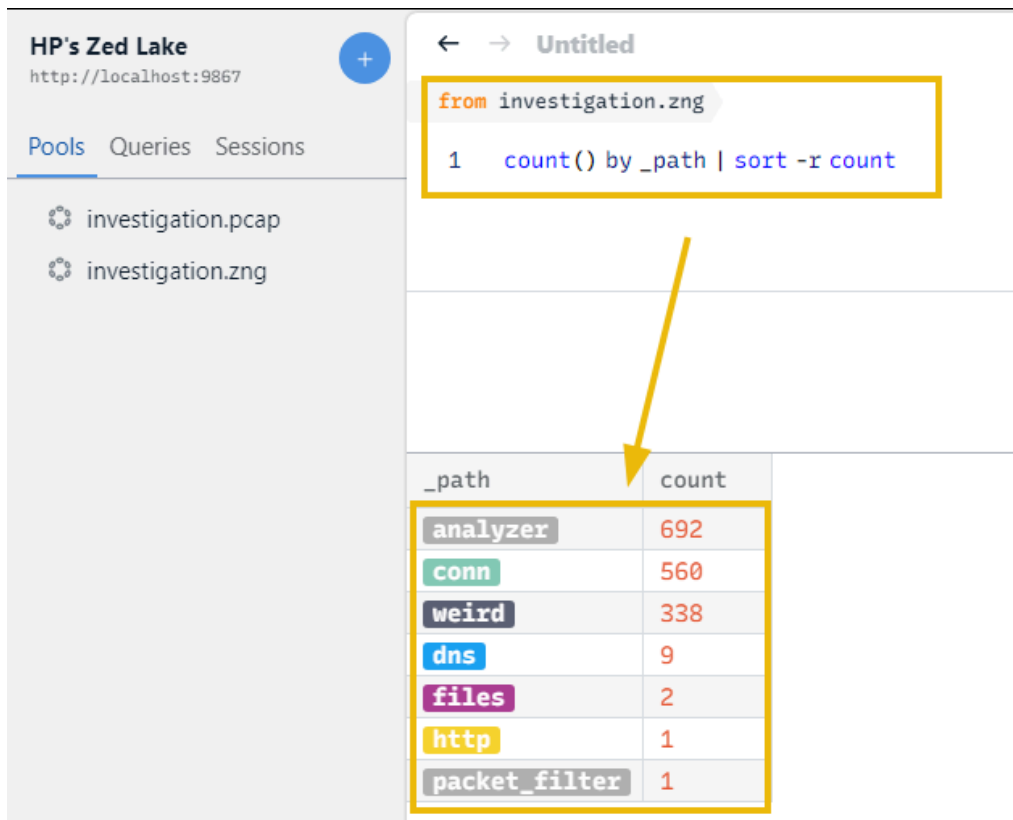
Component	Function
Pools panel (left)	Lists your loaded datasets and collections, allowing you to select which pool to query.
Query bar	The input field where you write Zed queries to filter, transform, and aggregate the active pool.
Results panel	Displays the matching logs in either a structured tabular view or raw record format.
Histogram	Shows a timeline of event frequency. You can click and drag to select and zoom into a time window.
Field list	Lists all active fields in the current results. Clicking a field automatically appends it to your query.
Saved queries	A library of bookmarked queries, allowing you to save your custom hunt logic for future use.

The nested `id` record stores the four-tuple connection. Zed allows you to query nested fields directly (e.g., `id.resp_h == 185.213.154.201`) without needing complex JOIN queries.

First Query: Counting Records: Select the `Investigation-Data` pool, from the left panel, click on Query Pool and run the following query in the query bar:

```
count() by _path | sort -r count
```

Query Explanation: This query groups the records by their log source path, counts the total occurrences for each type, and sorts the results in descending order.



The output shows `conn`, `weird`, `ssl`, `dns`, `files`, and `http` log types. The connection log (`conn`) dominates the counts because every session creates a connection record, whereas protocol logs are only created for specific protocol events.

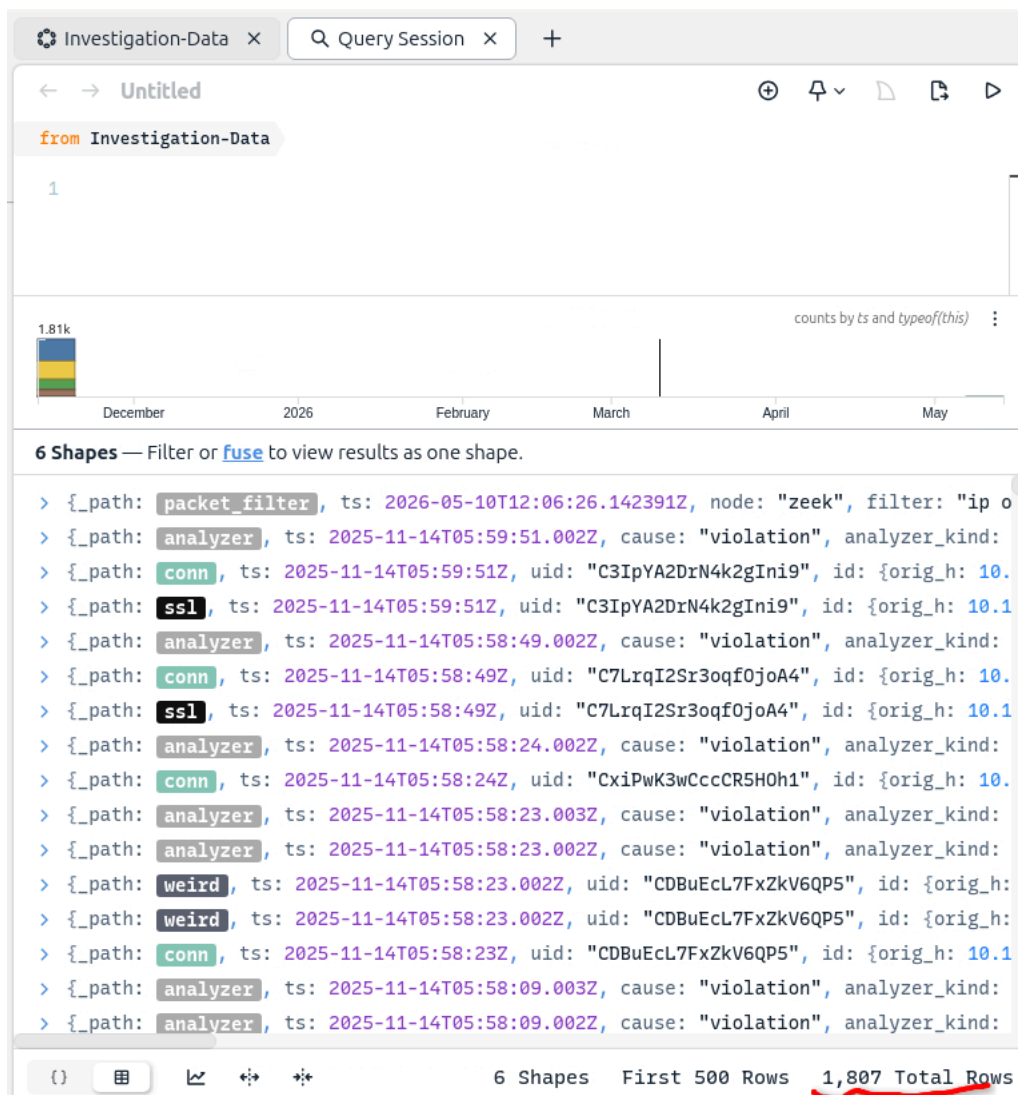
Using the Histogram: The histogram at the top provides a visual timeline of network activity. You can click and drag across any peak to zoom into a specific timeframe. Zui automatically appends a timestamp (`ts`) range filter to your query bar, narrowing down the results to only that active period.

Clicking Fields to Build Queries: You can quickly build queries without typing by clicking fields in the list. Clicking a field name like `id.resp_p` appends a group clause, while clicking a specific value in the results panel appends an equality filter. We will use this pivot-driven workflow throughout the room: running a baseline query, identifying an anomaly, and clicking it to pivot and dive deeper.

Answer the questions below

How many rows of results are captured in investigation-Data in Zui?

1807



Task 3 - The Zed Query Language

The Pipeline Model: Zed is pipeline-based. Each pipe symbol (|) takes the output of the preceding operation and passes it as input to the next. Queries are built from left to right: filtering first, then grouping, aggregating, sorting, and limiting.

```
_path == "conn" | not cidr_match(10.0.0.0/8, id.resp_h) | count() by id.resp_h | sort -r count | head 10
```

Query Explanation: Filters to `conn.log`, excludes internal destination subnets, counts connection frequency per external destination IP, sorts descending, and displays the top 10 results.

from Investigation-Data

```
1 _path == "conn" | not (id.resp_h in 10.0.0.0/8) | count() by id.resp_h | sort -r count | head 10
2
```

No data.

id	count
> {resp_h: 194.165.16.56}	202
> {resp_h: 13.107.6.152}	60
> {resp_h: 8.8.8.8}	56
> {resp_h: 1.1.1.1}	56
> {resp_h: 151.101.1.140}	55
> {resp_h: 142.250.190.46}	52
> {resp_h: 104.16.132.229}	41
> {resp_h: 10.14.0.53}	9
> {resp_h: 10.14.0.5}	4
> {resp_h: 10.14.22.120}	1

The destination IP `194.165.16.56` stands out with 202 connections, well ahead of the other CDN and resolver IPs. This single dominant destination is an anomaly that our query has surfaced for further investigation.

Basic Filtering: Any expression that evaluates to true or false can filter records. For example, to isolate connections going to port 443 (HTTPS), run:

```
_path == "conn" | id.resp_p == 443
```

from Investigation-Data

```
1 _path == "conn" | id.resp_p == 443
2
```

353

counts by ts and typeof(this)

	December	2026	February	March	April	May	
_path	ts	uid	id	proto	service	duration	orig_bytes
conn	2025-11-14T05:59:51Z	C3IPYA2DrN4k2gIni9	> {orig_h: 10.14.22.88, orig_p: 63672 ...+2 }	tcp	null	4ms	412
conn	2025-11-14T05:58:49Z	C7LrqI2Sr3oqf0JoA4	> {orig_h: 10.14.22.88, orig_p: 51540 ...+2 }	tcp	null	4ms	412
conn	2025-11-14T05:58:24Z	CxiPwK3wCccCR5H0h1	> {orig_h: 10.14.22.36, orig_p: 51842 ...+2 }	tcp	null	4ms	2827
conn	2025-11-14T05:57:51Z	Cb1L1u1EUx9xJICU3a	> {orig_h: 10.14.22.88, orig_p: 50687 ...+2 }	tcp	null	4ms	412
conn	2025-11-14T05:57:21Z	CVXXFu2EBAjRwvxDFh	> {orig_h: 10.14.22.136, orig_p: 49897 ...+2 }	tcp	null	4ms	1894
conn	2025-11-14T05:57:17Z	Cs7VY8gIMXJ7HqAf8	> {orig_h: 10.14.22.161, orig_p: 55562 ...+2 }	tcp	null	4ms	2638
conn	2025-11-14T05:56:50Z	CokXEm3JDWx9BmpvG9	> {orig_h: 10.14.22.88, orig_p: 61100 ...+2 }	tcp	null	4ms	412
conn	2025-11-14T05:55:49Z	CSbeq31s5i10xti1Tb	> {orig_h: 10.14.22.88, orig_p: 53007 ...+2 }	tcp	null	4ms	412
conn	2025-11-14T05:54:53Z	CcJzSsrrIQAwSALM8	> {orig_h: 10.14.22.88, orig_p: 63333 ...+2 }	tcp	null	4ms	412
conn	2025-11-14T05:53:49Z	CJhMhE1eRfZSKqMOE9	> {orig_h: 10.14.22.88, orig_p: 57527 ...+2 }	tcp	null	4ms	412
conn	2025-11-14T05:53:31Z	C218kL21kxw3jdp6v	> {orig_h: 10.14.22.36, orig_p: 50584 ...+2 }	tcp	null	4ms	2002

We can test whether an IP belongs to a subnet using the `cidr_match()` function. To isolate only external traffic by excluding all internal RFC 1918 private subnets, run the following filter:

```
_path == "conn" | not (cidr_match(10.0.0.0/8, id.resp_h) or cidr_match(172.16.0.0/12, id.resp_h) or cidr_match(192.168.0.0/16, id.resp_h))
```

from Investigation-Data

1

_path == "conn" | not (id.resp_h in [10.0.0.0/8, 172.16.0.0/12, 192.168.0.0/16])

2

3

560

counts by ts and typeof(this)

December

2026

February

March

April

May

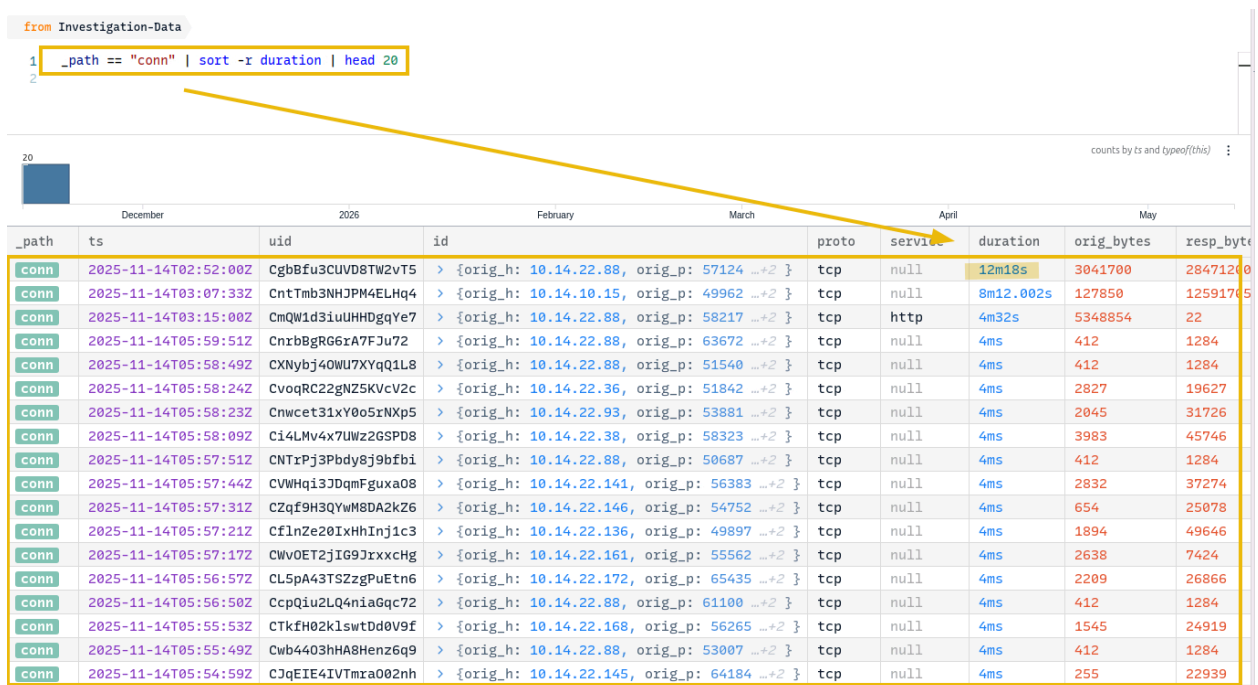
_path	ts	uid	id	proto	service	duration	orig_bytes
conn	2025-11-14T05:59:51Z	C3IPYA2DrN4k2gInI9	> {orig_h: 10.14.22.88, orig_p: 63672 ...+2 }	tcp	null	4ms	412
conn	2025-11-14T05:58:49Z	C7LrQI2Sr3oqf0JoA4	> {orig_h: 10.14.22.88, orig_p: 51540 ...+2 }	tcp	null	4ms	412
conn	2025-11-14T05:58:24Z	CxiPwK3wCccCR5H0h1	> {orig_h: 10.14.22.36, orig_p: 51842 ...+2 }	tcp	null	4ms	2827
conn	2025-11-14T05:58:23Z	CDBuEcL7FxZkV6QP5	> {orig_h: 10.14.22.93, orig_p: 53881 ...+2 }	tcp	null	4ms	2045
conn	2025-11-14T05:58:09Z	CVS7153THGELicbF42	> {orig_h: 10.14.22.38, orig_p: 58323 ...+2 }	tcp	null	4ms	3983
conn	2025-11-14T05:57:51Z	Cb1L1u1EUx9xJICU3a	> {orig_h: 10.14.22.88, orig_p: 50687 ...+2 }	tcp	null	4ms	412
conn	2025-11-14T05:57:44Z	C0Khhw2j1kHVWXCqx7	> {orig_h: 10.14.22.141, orig_p: 56383 ...+2 }	tcp	null	4ms	2832
conn	2025-11-14T05:57:31Z	CXLDI64naBJu340WV8	> {orig_h: 10.14.22.146, orig_p: 54752 ...+2 }	tcp	null	4ms	654
conn	2025-11-14T05:57:21Z	CVXXFu2EBAjRwvxDFh	> {orig_h: 10.14.22.136, orig_p: 49897 ...+2 }	tcp	null	4ms	1894
conn	2025-11-14T05:57:17Z	Cs7VY8g1MXJ7HqAf8	> {orig_h: 10.14.22.161, orig_p: 55562 ...+2 }	tcp	null	4ms	2638
conn	2025-11-14T05:56:57Z	C6qbc11ETT1PY0jIy4	> {orig_h: 10.14.22.172, orig_p: 65435 ...+2 }	tcp	null	4ms	2209
conn	2025-11-14T05:56:50Z	CokXEm3JDWx9BmpvG9	> {orig_h: 10.14.22.88, orig_p: 61100 ...+2 }	tcp	null	4ms	412
conn	2025-11-14T05:55:53Z	CiNFV23aifKIRnyeia	> {orig_h: 10.14.22.168, orig_p: 56265 ...+2 }	tcp	null	4ms	1545
conn	2025-11-14T05:55:49Z	CSbeq31s51i0xti1Tb	> {orig_h: 10.14.22.88, orig_p: 53007 ...+2 }	tcp	null	4ms	412
conn	2025-11-14T05:54:59Z	CjJLt02NrqiDVE01Yc	> {orig_h: 10.14.22.145, orig_p: 64184 ...+2 }	tcp	null	4ms	255
conn	2025-11-14T05:54:53Z	CeJzC0rrzIQAwcALMS	> {orig_h: 10.14.22.88, orig_p: 63333 ...+2 }	tcp	null	4ms	412

Query Explanation: Excludes all private destination subnets, ensuring we only see external traffic in our results. This filter is essential for threat hunting.

Sorting: By default, sorting is ascending. Use the `-r` flag with `sort` to reverse the order and sort in descending order. To surface the longest-running connections in the capture, run this query:

```
_path == "conn" | sort -r duration | head 20
```

Query Explanation: Sorts all connection records by their duration in descending order, displaying the top 20 longest sessions.

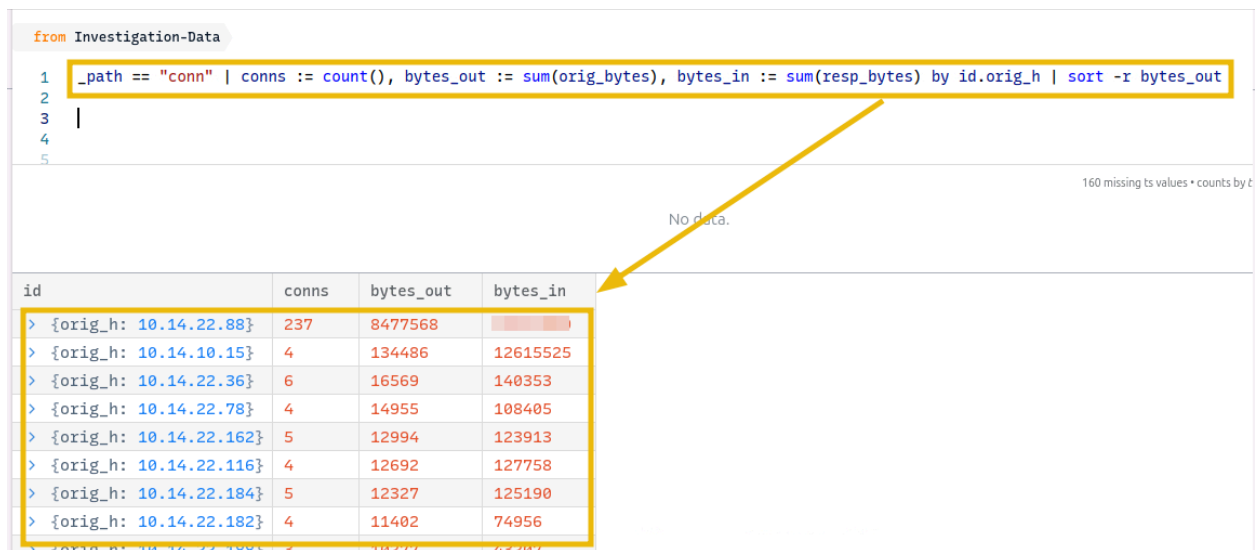


The results show a 738s RDP session and a 492s SMB session between internal hosts. Long-running sessions on administrative ports are baseline anomalies that deserve close inspection.

Aggregation: Aggregation allows you to summarise data. Use the syntax `<aggregator>() by <field>`. You can perform multiple calculations in parallel by defining aliases with the `:=` operator. To see a traffic volume profile per source IP, run:

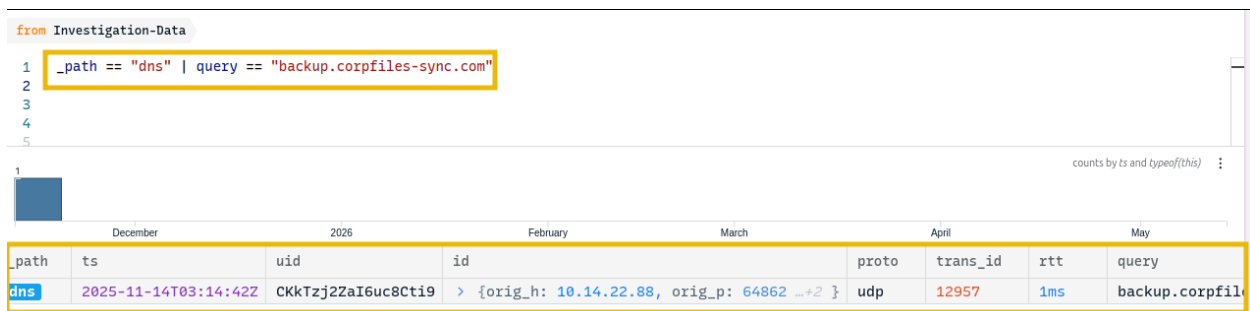
```
_path == "conn" | conns := count(), bytes_out := sum(orig_bytes), bytes_in := sum(resp_bytes) by id.orig_h | sort -r bytes_out
```

Query Explanation: Calculates the total connection count, total bytes sent, and total bytes received for each source IP, then sorts them by outbound data volume.



String and Pattern Matching: Three query patterns cover most string-matching threat hunts. To query for an exact domain name match, run:

```
_path == "dns" | query == "backup.corpfiles-sync.com"
```



To perform a substring search (useful for identifying domain lookalikes or typosquats), use the `grep()` function:

```
_path == "dns" | grep("corpfiles-sync", query)
```

```
from Investigation-Data
```

- `_path == "dns" | len(query) > 60`
-
-
-
-

counts by ts and typeof(this)

	December	2026	February	March		April	May
_path	ts	uid	id	proto	trans_id	rtt	query
dns	2025-11-14T05:37:00Z	CXRQ6X3odNIT6yWiVa	> {orig_h: 10.14.22.88, orig_p: 61865 ...+2 }	udp	33477	null	zm9yy2luz215a
dns	2025-11-14T05:08:00Z	Ccv1DHH2V19i2dPkf	> {orig_h: 10.14.22.88, orig_p: 53802 ...+2 }	udp	35712	null	dgfraw5nbwlzz
dns	2025-11-14T04:42:00Z	CmK2074G0r0RHxqjYg	> {orig_h: 10.14.22.88, orig_p: 52280 ...+2 }	udp	2580	null	c2v1bwluz2x5c
dns	2025-11-14T04:13:00Z	CgSMuqH0ZD9W9fOc	> {orig_h: 10.14.22.88, orig_p: 53372 ...+2 }	udp	53087	null	zgfvdxtzgf0y
dns	2025-11-14T03:48:00Z	Ch9k0qky1pHXpvWfK	> {orig_h: 10.14.22.88, orig_p: 56847 ...+2 }	udp	62154	null	bm9ybwfsbg9va
dns	2025-11-14T03:22:00Z	CdkkfQ3iisvWVtZR13	> {orig_h: 10.14.22.88, orig_p: 56967 ...+2 }	udp	8081	null	dghpc2lzyxr1c
dns	2025-11-14T02:55:00Z	CANoH5WFasehRmFXc	> {orig_h: 10.14.22.88, orig_p: 55062 ...+2 }	udp	60413	null	agvsbe93b3is2

To hunt for DNS tunnelling signatures, we can search for queries longer than 60 characters by measuring query string length:

```
_path == "dns" | len(query) > 60
```

```
from Investigation-Data
```

- `_path == "dns" | len(query) > 60`
-
-
-
-

counts by ts and typeof(this)

	December	2026	February	March		April	May
_path	ts	uid	id	proto	trans_id	rtt	query
dns	2025-11-14T05:37:00Z	CXRQ6X3odNIT6yWiVa	> {orig_h: 10.14.22.88, orig_p: 61865 ...+2 }	udp	33477	null	zm9yy2luz215a
dns	2025-11-14T05:08:00Z	Ccv1DHH2V19i2dPkf	> {orig_h: 10.14.22.88, orig_p: 53802 ...+2 }	udp	35712	null	dgfraw5nbwlzz
dns	2025-11-14T04:42:00Z	CmK2074G0r0RHxqjYg	> {orig_h: 10.14.22.88, orig_p: 52280 ...+2 }	udp	2580	null	c2v1bwluz2x5c
dns	2025-11-14T04:13:00Z	CgSMuqH0ZD9W9fOc	> {orig_h: 10.14.22.88, orig_p: 53372 ...+2 }	udp	53087	null	zgfvdxtzgf0y
dns	2025-11-14T03:48:00Z	Ch9k0qky1pHXpvWfK	> {orig_h: 10.14.22.88, orig_p: 56847 ...+2 }	udp	62154	null	bm9ybwfsbg9va
dns	2025-11-14T03:22:00Z	CdkkfQ3iisvWVtZR13	> {orig_h: 10.14.22.88, orig_p: 56967 ...+2 }	udp	8081	null	dghpc2lzyxr1c
dns	2025-11-14T02:55:00Z	CANoH5WFasehRmFXc	> {orig_h: 10.14.22.88, orig_p: 55062 ...+2 }	udp	60413	null	agvsbe93b3is2

Cut: Selecting Specific Fields: The `cut` operator allows you to select only the specific fields you want to view, cleaning up your results panel. Run the following to focus only on connection metadata for the compromised host:

```
_path == "conn" | id.orig_h == 10.14.22.88 | cut ts, id.resp_h, id.resp_p, duration, orig_bytes, resp_bytes
```

```
from Investigation-Data
1  _path == "conn" | id.orig_h == 10.14.22.88 | cut ts, id.resp_h, id.resp_p, duration, orig_bytes, resp_bytes
2
3
4
5
```

237



December		2026	February	March	April
ts	id	duration	orig_bytes	resp_bytes	
2025-11-14T05:59:51Z	> {resp_h: 194.165.16.56, resp_p: 443}	4ms	412	1284	
2025-11-14T05:58:49Z	> {resp_h: 194.165.16.56, resp_p: 443}	4ms	412	1284	
2025-11-14T05:57:51Z	> {resp_h: 194.165.16.56, resp_p: 443}	4ms	412	1284	
2025-11-14T05:56:50Z	> {resp_h: 194.165.16.56, resp_p: 443}	4ms	412	1284	
2025-11-14T05:55:49Z	> {resp_h: 194.165.16.56, resp_p: 443}	4ms	412	1284	
2025-11-14T05:54:53Z	> {resp_h: 194.165.16.56, resp_p: 443}	4ms	412	1284	
2025-11-14T05:53:49Z	> {resp_h: 194.165.16.56, resp_p: 443}	4ms	412	1284	
2025-11-14T05:52:48Z	> {resp_h: 194.165.16.56, resp_p: 443}	4ms	412	1284	
2025-11-14T05:51:50Z	> {resp_h: 194.165.16.56, resp_p: 443}	4ms	412	1284	
2025-11-14T05:50:55Z	> {resp_h: 194.165.16.56, resp_p: 443}	4ms	412	1284	
2025-11-14T05:49:57Z	> {resp_h: 194.165.16.56, resp_p: 443}	4ms	412	1284	
2025-11-14T05:48:54Z	> {resp_h: 194.165.16.56, resp_p: 443}	4ms	412	1284	

Cross-Log Pivoting via UID: Every Zeek log record associated with a single TCP session shares a unique connection ID called a `uid`. Pivoting by `uid` allows you to track an event across multiple log types without needing database joins.

```
_path == "conn" | id.resp_h == 185.213.154.201
```

from Investigation-Data

1 _path == "conn" | id.resp_h == 185.213.154.201

2

3

4

5

counts by ts and typeof(this)

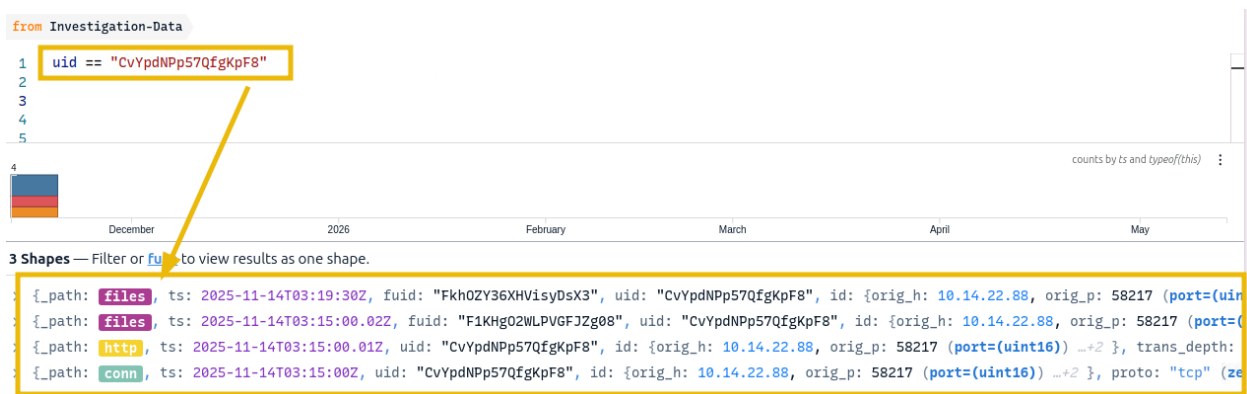
1



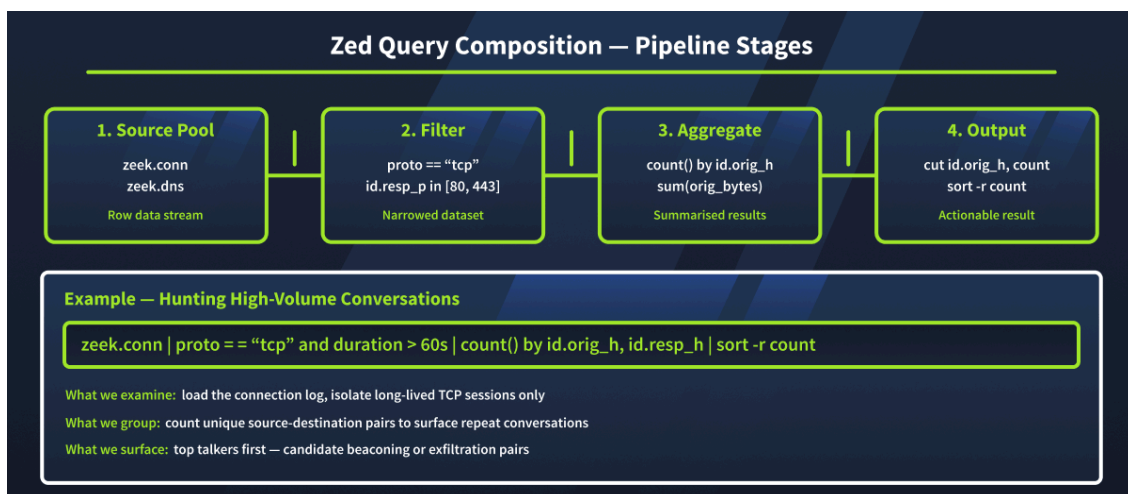
December		2026	February	March	April	May	
path	ts	uid	id	proto	service	duration	orig bytes
conn	2025-11-14T03:15:00Z	CvYpdNpp57QfgKpF8	> {orig_h: 10.14.22.88, orig_p: 58217 ...+2 }	tcp	http	4m32s	5348854

Select the resulting row in the panel and copy the `uid` value (the canonical ID for this session is `CvYpdNpp57QfgKpF8`). Now, query by this UID without any path filter to see every log record generated by this session:

```
uid == "CvYpdNpp57QfgKpF8"
```



Query Explanation: Returns all Zeek logs associated with this session, including TCP connection details, HTTP request headers, and file transfer logs.



Building a Hunt Query Step by Step: We will filter for external HTTPS traffic, calculate connection counts and outbound bytes grouped by source and destination, and filter for pairs with high session volumes:

```
_path == "conn" | id.resp_p == 443 | not (cidr_match(10.0.0.0/8, id.resp_h) or cidr_match(172.16.0.0/12, id.resp_h) or cidr_match(192.168.0.0/16, id.resp_h)) | sessions := count(), bytes_out := sum(orig_bytes) by id.orig_h, id.resp_h | sessions >= 20 | sort -r sessions
```

from Investigation-Data

1 _path == "conn" | id.resp_p == 443 | not (id.resp_h in [10.0.0.0/8, 172.16.0.0/12, 192.168.0.0/16]) | sessions := count(), bytes_out := sum(orig

2

3

4

5

1 missing ts values • counts by ts and typeof(this) :

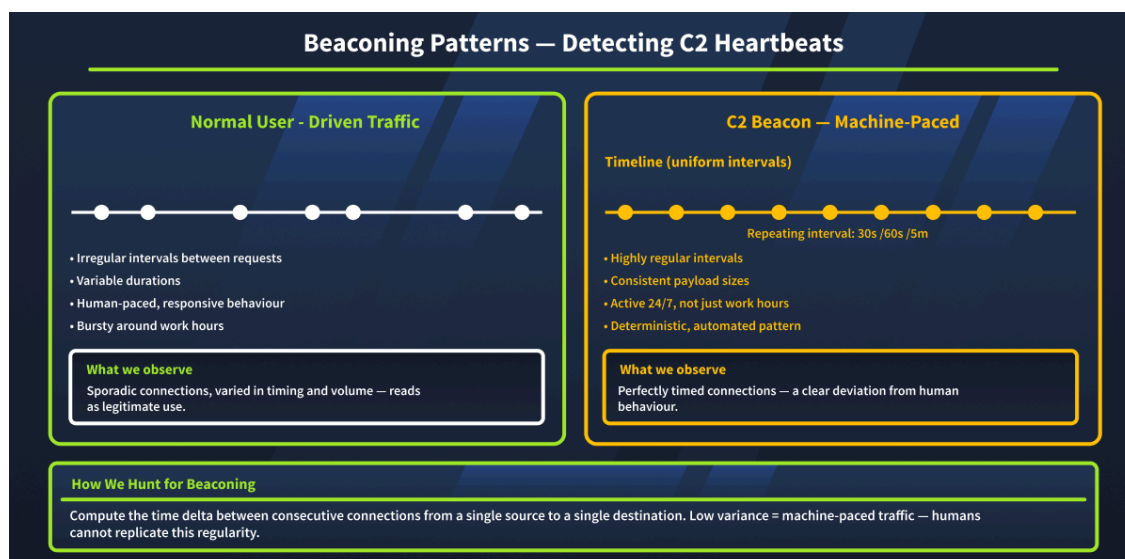
No data.

id	sessions	bytes_out
> {orig_h: 10.14.22.88, resp_h: 194.165.16.56}	202	83224

Query Explanation: Lists source-destination IP pairs where the client opened 20 or more external HTTPS connections, sorting by the connection frequency to identify candidate C2 beacons.

Task 4 - Hunting for Beaconsing Patterns

Our hypothesis is that an endpoint is compromised and running malware that communicates with a Command and Control (C2) server at regular intervals. Standard signature-based rules did not fire because each connection is small and encrypted. The hunting question is: what does regular-interval repetition look like at the log level?



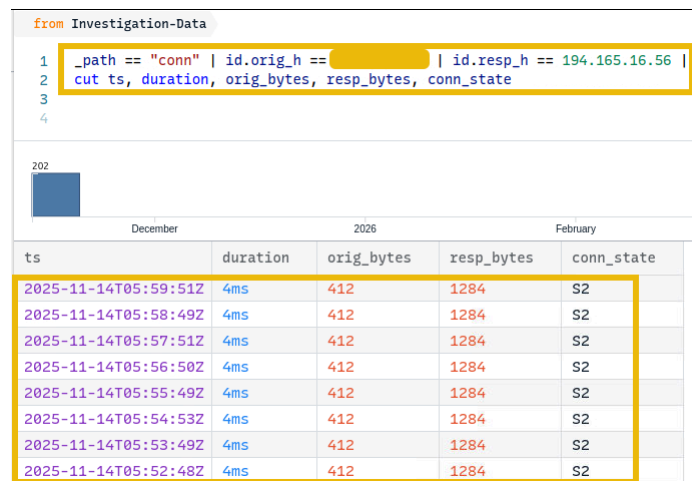
What Beaconsing Looks Like: A beaconsing implant produces `conn.log` records with these characteristics:

- Multiple connections from the same source to the same destination
- Consistent or near-consistent intervals (60s, 300s, 3600s common, with 10-30 percent jitter)
- Short duration per connection (typically under 10 seconds)
- Low data volume per connection (a few hundred bytes each direction)
- High reliability (hundreds per day, not sporadic)

Step 1: Identify High-Frequency External Connections: Let us run this query to surface high-frequency external connections:

```
_path == "conn" |  
not (cidr_match(10.0.0.0/8, id.resp_h) or cidr_match(172.16.0.0/12, id.resp_h) or cidr_match(192.168.0.0/16, id.  
resp_h)) |  
count() by id.orig_h, id.resp_h, id.resp_p |  
sort -r count |  
count > 20
```

Query Explanation: Counts connections grouped by source IP, destination IP, and port, then filters to tuples that exceed 20 connections.

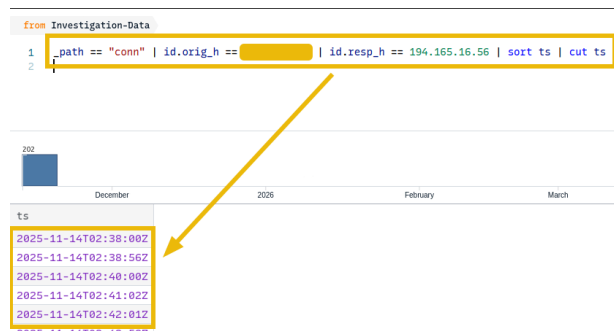


The results show 202 connections from a single host to the destination IP on port 443, which is well outside normal browsing baselines. The Fastly row underneath is typical CDN background noise.

Step 2: Examine the Interval Pattern: Let us pull the timestamps to see how regularly those connections arrive:

```
_path == "conn" | id.orig_h == 10.14.22.88 | id.resp_h == 194.165.16.56 | sort ts | cut ts
```

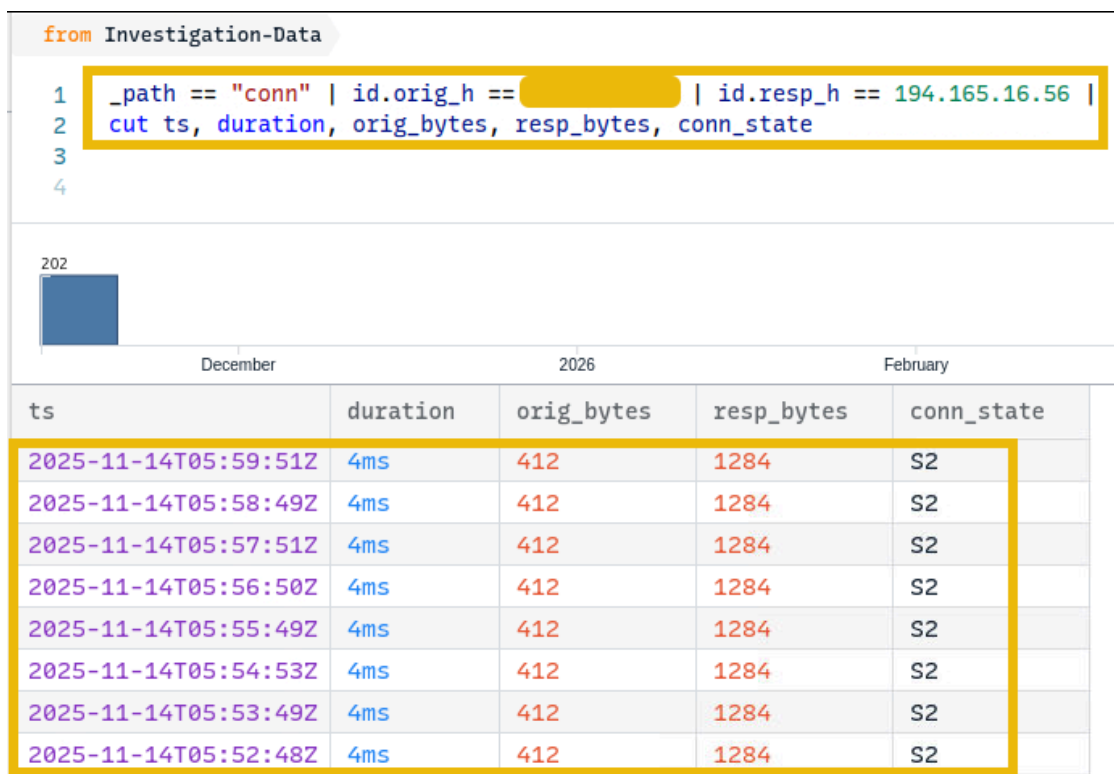
Query Explanation: Returns timestamps for all connections from the suspect to the candidate C2, sorted chronologically. The histogram above the results panel shows evenly spaced spikes.



Notice that the connections land roughly 60 seconds apart with slight variations (jitter), which is the signature of a Cobalt Strike C2 beacon. Legitimate polling services show perfectly fixed intervals (cron-like) or highly random patterns (user-driven).

Step 3: Examine Connection Characteristics: Now let's look at the byte counts and connection state for each beacon session:

```
_path == "conn" | id.orig_h == <REDACTED> | id.resp_h == 194.165.16.56 |  
cut ts, duration, orig_bytes, resp_bytes, conn_state
```



```
from Investigation-Data  
1 _path == "conn" | id.orig_h == <REDACTED> | id.resp_h == 194.165.16.56 |  
2 cut ts, duration, orig_bytes, resp_bytes, conn_state  
3  
4
```

ts	duration	orig_bytes	resp_bytes	conn_state
2025-11-14T05:59:51Z	4ms	412	1284	S2
2025-11-14T05:58:49Z	4ms	412	1284	S2
2025-11-14T05:57:51Z	4ms	412	1284	S2
2025-11-14T05:56:50Z	4ms	412	1284	S2
2025-11-14T05:55:49Z	4ms	412	1284	S2
2025-11-14T05:54:53Z	4ms	412	1284	S2
2025-11-14T05:53:49Z	4ms	412	1284	S2
2025-11-14T05:52:48Z	4ms	412	1284	S2

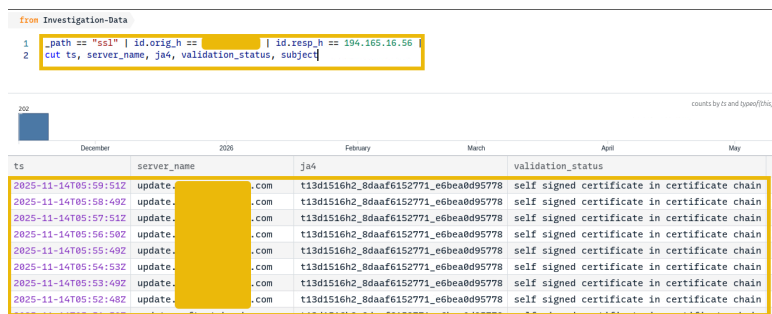
Every session carries 412 bytes outbound and 1,284 bytes inbound. Three properties confirm structured traffic:

- **Identical byte counts.** 412 out / 1,284 in is the mathematical signature of a fixed C2 beacon protocol.
- **Consistent duration.** Consistency matters more than the exact value; user sessions show highly varied durations.
- **Identical conn_state.** Real user traffic shows a mix (SF, RSTO, S1, OTH); a beacon shows a single state repeatedly.

Step 4: Correlate With TLS Logs: Let us pivot to `ssl.log` to pull the TLS handshake metadata for these connections:

```
_path == "ssl" | id.orig_h == <REDACTED> | id.resp_h == 194.165.16.56 |
cut ts, server_name, ja4, validation_status, subject
```

Query Explanation: Returns TLS handshake metadata for every beacon connection: SNI, JA4 client fingerprint, certificate validation result, and certificate subject.



Three indicators identify the C2 framework with high confidence:

- SNI `update.<REDACTED>.com` : typosquat, using a plausible update domain.
- Self-signed cert with subject `CN=localhost` : the default Cobalt Strike profile.
- JA4 fingerprint `t13d1516h2_8daaf6152771_e6bea0d95778` : cross-references the [public JA4 catalogue](#) (opens in new tab) to confirm the Cobalt Strike default profile.

Answer the questions below

Run the beaconing hunt query; what is the source IP of the host with the highest external connection count to a single destination?

10.14.22.88

run the query below:

```
"conn" | not (id.resp_h in [10.0.0.0/8, 172.16.0.0/12, 192.168.0.0/16]) | count() by id.orig_h, id.resp_h, id.re
sp_p | sort -r count | count>20
```

In Zeek, `conn` represents the baseline log that records summary metadata for every network connection attempt. This query finds internal IP addresses making more than 20 connection attempts to external destination IP and port combinations.

Investigation-Data

```

1 "conn" | not (id.resp_h in [10.0.0.0/8, 192.168.0.0/16]) | count() by id.orig_h, id.resp_h, id.resp_p
2 192.168.0.0/16 | count() by id.orig_h, id.resp_h, id.resp_p
3 | sort -r count | count>20 |

```

1 missing ts values • counts by ts and typ

No data.

id	count
> {orig_h: 10.14.22.88, resp_h: 194.165.16.56 ...+1 }	404

What TLS server name is associated with the beaconing destination?

update.softpatch-cdn.com

run the query below:

```

_path == "ssl" | id.orig_h == 10.14.22.88 | id.resp_h == 194.165.16.56 | cut ts, server_name, ja4, validation_status, subject

```

This query retrieves specific TLS certificate and metadata fields for encrypted connections between source IP **10.14.22.88** and destination IP **194.165.16.56**.

Investigation-Data

```

1 _path == "ssl" | id.orig_h == 10.14.22.88
2 | id.resp_h == 194.165.16.56
3 | cut ts, server_name, ja4, validation_status, subject

```

counts by ts and typeof(this)

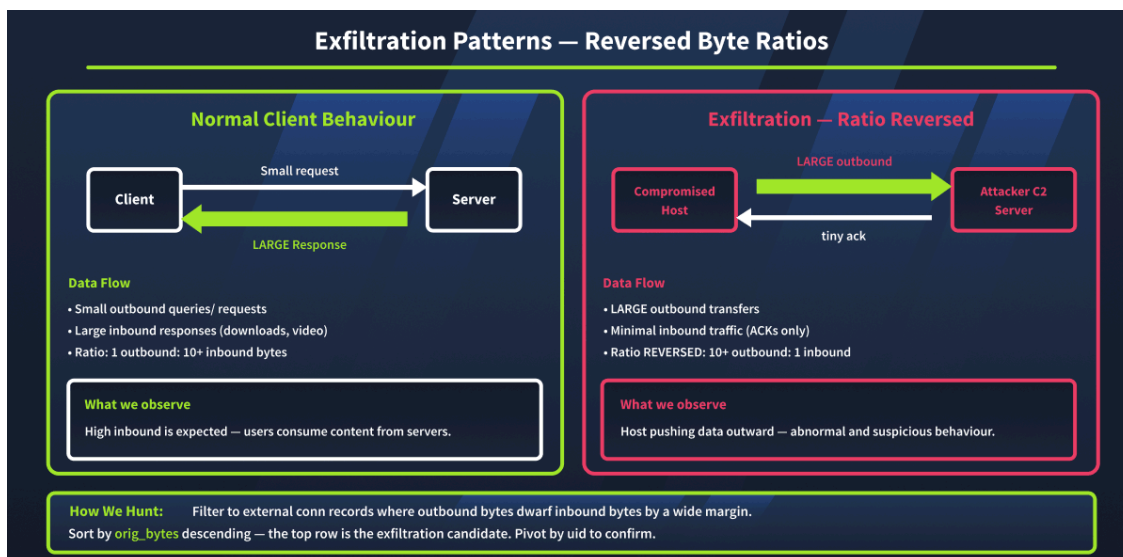
202

December 2026 February March April May

ts	server_name	ja4
2025-11-14T05:59:51Z	update.softpatch-cdn.com	t13d1516h2_8daaf6152771
2025-11-14T05:58:49Z	update.softpatch-cdn.com	t13d1516h2_8daaf6152771
2025-11-14T05:57:51Z	update.softpatch-cdn.com	t13d1516h2_8daaf6152771
2025-11-14T05:56:50Z	update.softpatch-cdn.com	t13d1516h2_8daaf6152771
2025-11-14T05:55:49Z	update.softpatch-cdn.com	t13d1516h2_8daaf6152771

Task 5 - Hunting Data Staging and Exfiltration

Our next hypothesis is that an attacker who has achieved persistence is preparing to exfiltrate collected data. We want to find the outbound transfer event, identify its destination, and tie the network evidence back to the uploaded file. Unlike C2 beaconing, data exfiltration often produces a small number of sessions with unusually high outbound volume.

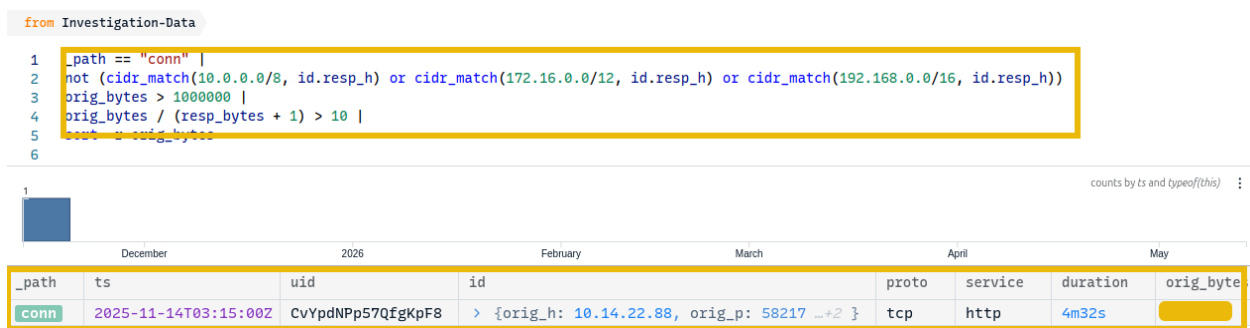


Step 1: Find Asymmetric External Transfers

Normal web browsing is asymmetric: clients usually receive far more data than they send. A client that sends a large amount of data to an external host while receiving very little back is worth investigating. Run this query to isolate external connections where the source sent more than 1 MB and the outbound-to-inbound ratio is greater than 10:1:

```
_path == "conn" |
not (cidr_match(10.0.0.0/8, id.resp_h) or cidr_match(172.16.0.0/12, id.resp_h) or cidr_match(192.168.0.0/16, id.resp_h)) |
orig_bytes > 1000000 |
orig_bytes / (resp_bytes + 1) > 10 |
sort -r orig_bytes
```

Query Explanation: The private-network checks remove internal destinations, `orig_bytes > 1000000` keeps only sizeable uploads, and `resp_bytes + 1` prevents division-by-zero errors when the response byte count is empty or zero.



Focus on the top row. It gives us the source host, the external destination IP, the protocol, the service, and the outbound byte count. The connection log byte count includes TCP and HTTP overhead, so it can be slightly larger than the raw uploaded file size we will see in protocol-specific logs.

Step 2: Check the Timing

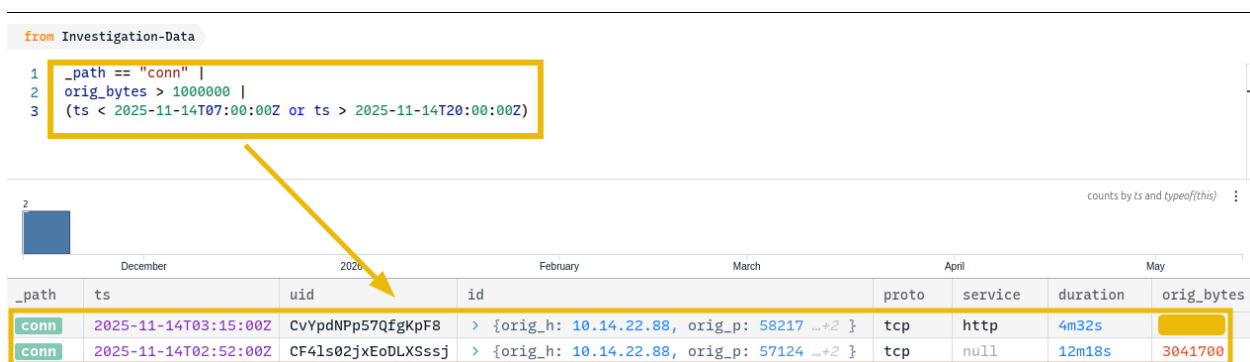
Large transfers are more suspicious when they happen outside normal working hours. Run this query to check for transfers larger than 1 MB before 07:00 UTC or after 20:00 UTC:

```

_path == "conn" |
orig_bytes > 1000000 |
(ts < 2025-11-14T07:00:00Z or ts > 2025-11-14T20:00:00Z)

```

Query Explanation: This uses direct timestamp comparison, which is portable in this lab's Zed build. The result should be compared with the destination and byte-count evidence from Step 1.



The same large transfer occurs during off-hours. Off-hours activity alone is not proof of exfiltration, because backup jobs and maintenance tasks can also run overnight. It does, however, give us a reason to pivot into the application-layer records for that destination.

Step 3: Pivot into HTTP Evidence

The suspicious connection uses plain HTTP, so Zeek can extract request metadata from `http.log`. Use the external destination IP from the top row of Step 1 as the pivot:

```

_path == "http" |
id.resp_h == 185.213.154.201 |

```

Query Explanation: This filters HTTP records for the external destination, sorts by uploaded body size, and keeps the fields needed to explain the upload: method, host, URI, body length, MIME type, and User-Agent.

```
1 _path == "http" |
2 id.resp_h == 185.213.154.201 |
3 sort -r request_body_len |
4 cut ts, id.orig_h, id.resp_h, method, host, uri, request_body_len, orig_mime_types, user_agent
```

This is where the domain enters the investigation. The `host` field tells us the upload was sent to `backup.corpfiles-sync.com`. The same row also shows a POST to `<REDACTED>`, a ZIP MIME type, and a blank `User-Agent`. A missing User-Agent is unusual for a normal browser and often points to a script or custom client.

Now that the hostname has been discovered from HTTP evidence, use it as the DNS pivot. This checks whether the source host resolved that domain shortly before the upload:

Query Explanation: The DNS query uses the hostname observed in `http.log`. Filtering on `query` keeps the lookup easy to read and avoids depending on version-specific handling of the `answers` field.

```
1 _path == "dns" |
2 query == "backup.corpfiles-sync.com" |
3 cut ts, id.orig_h, query, answers, rcode_name
```

L2 - 12.4. Threat Hunting with Zui

The DNS lookup occurred less than a minute before the HTTP POST. That gives us a clean chain: the host resolved the domain, connected to the returned IP, and uploaded a ZIP payload over HTTP.

Step 5: File Extraction and Hashing

Zeek's `files.log` records transferred files across supported protocols. The `tx_hosts` and `rx_hosts` fields are sets, so use the `in` operator rather than equality. Run this query using the source and destination from the exfiltration connection:

```
_path == "files" |
10.14.22.88 in tx_hosts |
185.213.154.201 in rx_hosts |
cut ts, tx_hosts, rx_hosts, conn_uids, source, analyzers, mime_type, filename, total_bytes, md5, sha1
```

Query Explanation: This returns every file record where the suspected source appears in the sender set and the exfiltration destination appears in the receiver set. The `cut` command keeps the file evidence fields visible, including filename, size, MIME type, and the hash values populated by the file analyzers. In this dataset, the `analyzers` field shows `MD5` and `SHA1`, so those are the hashes visible in Zui.



The screenshot shows the Zui interface. At the top, a query is entered in a text box:

```
from Investigation-Data
1 _path == "files" |
2 10.14.22.88 in tx_hosts |
3 185.213.154.201 in rx_hosts
```

Below the query, a table of results is displayed. The table has columns: ts, tx_hosts, rx_hosts, conn_uids, source, depth, analyzers, mime_type, filename, duration, and location. The first row of data is highlighted in yellow, corresponding to the query results:

ts	tx_hosts	rx_hosts	conn_uids	source	depth	analyzers	mime_type	filename	duration	location
83	[185.213.154.201]	[10.14.22.88]	[cvrpdNp57qfgKpF8]	HTTP	0	[MD5, SHA1]	application/zip	[redacted]	57.3s	tru

The file record confirms the transferred object. The archive filename, MIME type, size, MD5, and SHA1 can now be used as incident response evidence. This information is useful because it gives the response team concrete IOCs to search for across hosts, file stores, and endpoint telemetry.

Staging: The Two-Hop Pattern

A high-volume internal SMB transfer can be a precursor to staged exfiltration. Run this query to find internal SMB sessions where one host pulled more than 10 MB:

```
_path == "conn" |
(cidr_match(10.0.0.0/8, id.orig_h) or cidr_match(172.16.0.0/12, id.orig_h) or cidr_match(192.168.0.0/16, id.orig_h)) |
(cidr_match(10.0.0.0/8, id.resp_h) or cidr_match(172.16.0.0/12, id.resp_h) or cidr_match(192.168.0.0/16, id.resp_h)) |
id.resp_p == 445 |
resp_bytes > 10000000 |
sort -r resp_bytes
```

from Investigation-Data

```
1  _path == "conn" |
2  (cidr_match(10.0.0.0/8, id.orig_h) or cidr_match(172.16.0.0/12, id.orig_h) or cidr_match(192.168.0.0/16, id.orig_h)) |
3  (cidr_match(10.0.0.0/8, id.resp_h) or cidr_match(172.16.0.0/12, id.resp_h) or cidr_match(192.168.0.0/16, id.resp_h)) |
4  id.resp_p == 445 |
5  resp_bytes > 10000000 |
6  sort -r resp_bytes
```

1

December		2026	February		March	April		
_path	ts	uid	id	proto	service	duration	orig_bytes	resp_bytes
conn	2025-11-14T03:07:33Z	CkJbLp4FEQ4Vce48Gb	> {orig_h: 10.14.10.15, orig_p: 49962 ...}	tcp	null	8m12.002s	127850	12591705

The result shows an internal SMB transfer into the IT admin workstation before the outbound upload. Let us take a note of this point, as the next task follows that lateral movement chain and explains how the attacker reached the administrative segment.

Answer the questions below

What is the orig_bytes value for the largest external upload?

5348854

orig_bytes

```

_path == "conn " | not (cidr_match(10.0.0.0/8, id.resp_h) or cidr_match(172.16.0.0/12, id.resp_h) or cidr_match(192.168.0.0/16, id.resp_h)) | orig_bytes / (resp_bytes + 1) > 10 | sort -r orig_bytes

```

This query identifies outbound network connections to non-private destination IPs where the client sent significantly more data than it received, sorted by total bytes sent to highlight potential data exfiltration.

What URI was used for the HTTP POST to the exfiltration endpoint?

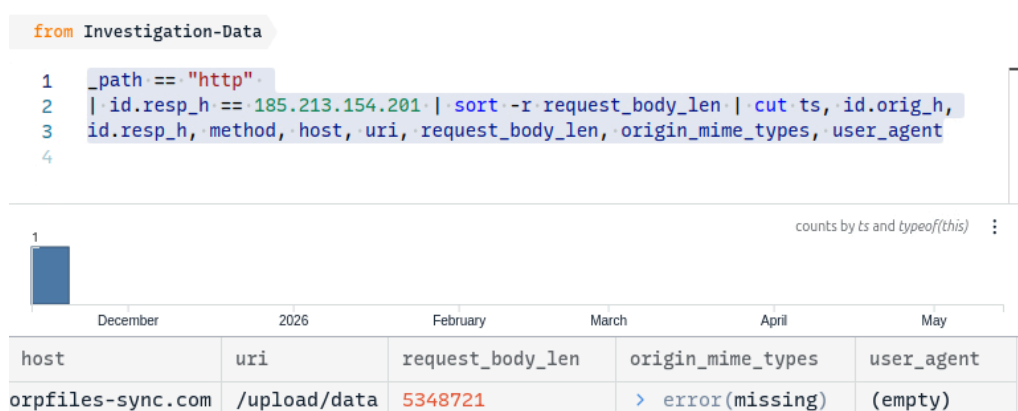
/upload/data

```

_path == "http"
| id.resp_h == 185.213.154.201 | sort -r request_body_len | cut ts, id.orig_h,
id.resp_h, method, host, uri, request_body_len, origin_mime_types, user_agent

```

This query lists HTTP requests made to IP `185.213.154.201`, showing request details and payload sizes sorted largest to smallest to highlight potential data uploads or web attacks.



What is the filename of the archive POSTed to 185.213.154.201 over plain HTTP?

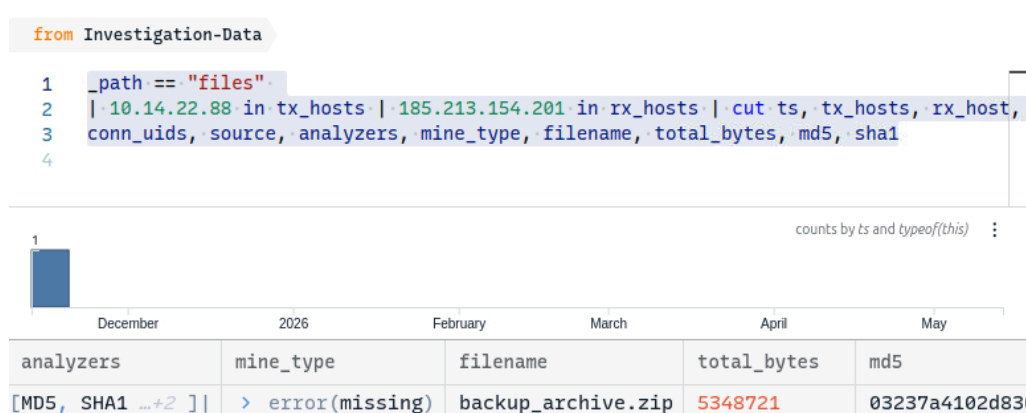
backup_archive.zip

```

_path == "files"
| 10.14.22.88 in tx_hosts | 185.213.154.201 in rx_hosts | cut ts, tx_hosts, rx_host,
conn_uids, source, analyzers, mine_type, filename, total_bytes, md5, sha1

```

This query extracts file transfer metadata, hash signatures, and transfer protocol details for files sent from host 10.14.22.88 to destination 185.213.154.201.



What is the MD5 hash value of the ZIP file downloaded by the Finance workstation?

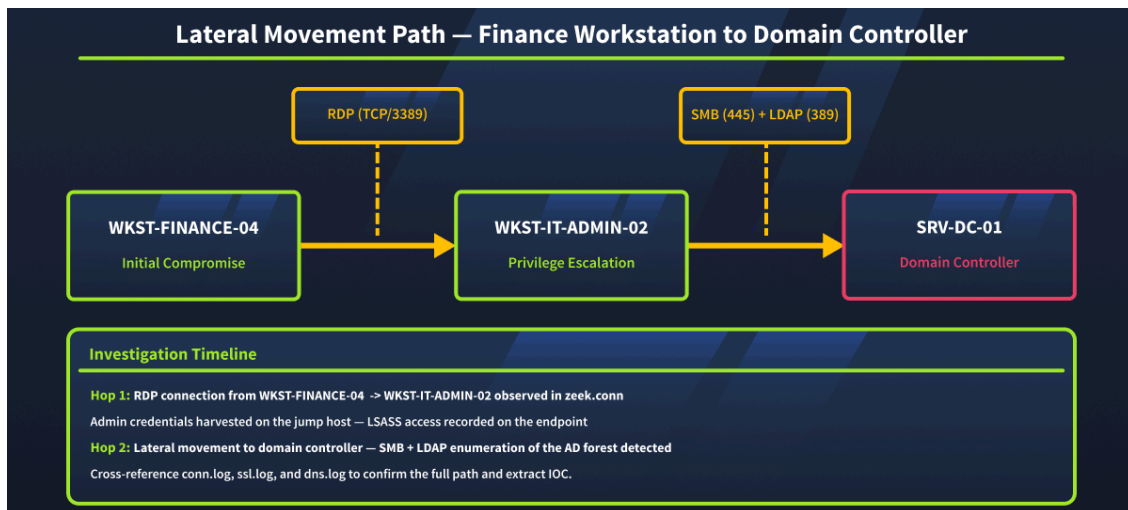
03237a4102d830519fa0dfb47c88c467

Task 6 - Hunting for Lateral Movement

Lateral movement shows how the attacker expanded their access from the initial foothold toward higher-value administrative targets. The network signature: internal-to-internal connections on administrative ports (such as SMB or RDP) initiated by a workstation without a legitimate administrative role.

Hypothesis and Patterns: Our compromised host is `10.14.22.88` (`WKST-FINANCE-04`), a Finance subnet workstation. Since Finance workstations do not run administrative scripts or connect to other clients on system management ports, this host should never initiate such connections. Observable patterns to hunt for include:

- **Administrative port connections:** SMB (445), RDP (3389), WinRM (5985/5986), SSH (22)
- **Scanning signatures:** many `S0` (SYN-only) connections from one source to many destinations
- **Authenticated sessions with data movement:** full handshake, then KB-to-MB transfer
- **Sequential pivots:** A to B, then B to C, leaving a chain that we can reconstruct



Step 1: Unexpected Internal Administrative Connections: Let us run this query to surface internal admin port activity:

```

_path == "conn" |
(cidr_match(10.0.0.0/8, id.orig_h) or cidr_match(172.16.0.0/12, id.orig_h) or cidr_match(192.168.0.0/16, id.orig_h)) |
(cidr_match(10.0.0.0/8, id.resp_h) or cidr_match(172.16.0.0/12, id.resp_h) or cidr_match(192.168.0.0/16, id.resp_h)) |
id.resp_p in [445, 3389, 5985, 22, 135, 389] |
not id.orig_h == id.resp_h |
count() by id.orig_h, id.resp_p |
sort -r count
  
```

Query Explanation: Internal administrative connections (SMB 445, RDP 3389, WinRM 5985, SSH 22, RPC 135, LDAP 389), grouped by source and port.

The output:

from Investigation-Data

```
2 (cidr_match(10.0.0.0/8, id.orig_h) or cidr_match(172.16.0.0/12, id.orig_h) or cidr_match(1
3 (cidr_match(10.0.0.0/8, id.resp_h) or cidr_match(172.16.0.0/12, id.resp_h) or cidr_match(1
4 id.resp_p in [445, 3389, 5985, 22, 135, 389] |
5 not id.orig_h == id.resp_h |
6 count() by id.orig_h, id.resp_p |
```

4 missing to values + counts by to and type of (this)

No data.

id	count
> {orig_h: 10.14.22.88, resp_p: 445}	
> {orig_h: 10.14.10.15, resp_p: 389}	3
> {orig_h: 10.14.10.15, resp_p: 445}	1
> {orig_h: 10.14.22.88, resp_p: 3389}	1

The results show 20+ SMB connection attempts from 10.14.22.88, which is highly anomalous. A single RDP session initiated by this Finance workstation, along with subsequent LDAP/SMB activity from the target host (10.14.10.15), is suspicious.

Step 2: Detect the Scan: `conn_state == "S0"` means "SYN sent, no reply". Many S0 records from a single source on a single port indicate an internal port scan. Let us use the following query to confirm:

```
_path == "conn" | id.orig_h == 10.14.22.88 | id.resp_p == 445 | conn_state == "S0" | count()
```

from Investigation-Data

```
1 _path == "conn" | id.orig_h == 10.14.22.88 | id.resp_p == 445 | conn_state == "S0" | count()
```

No data.

(uint64)

The count confirms 20+ unanswered connection attempts. The attacker scanned 20+ hosts on port 445 (indicated by the S0 state) during discovery, and then pivoted via RDP to the IT admin workstation (10.14.10.15), which they identified as active.

Step 3: The First Pivot via RDP: Let us look at the RDP session that followed the scan:

```
_path == "conn" | id.orig_h == 10.14.22.88 | id.resp_p == 3389 | sort ts |
cut ts, id.resp_h, duration, orig_bytes, resp_bytes
```

The output:

from Investigation-Data

```
1 _path == "conn" | id.orig_h == 10.14.22.88 | id.resp_p == 3389 | sort ts |
2 cut ts, id.resp_h, duration, orig_bytes, resp_bytes
```

1					
					
	December2026FebruaryMarch				
ts	id	duration	orig_bytes	resp_bytes	
2025-11-14T02:52:00Z	> {resp_h: 10.14.10.15}	12m18s	3041700	28471200	

The output reveals a single RDP connection from 10.14.22.88 to 10.14.10.15, lasting 738 seconds (about 12 minutes). This result identifies the first-hop workstation, not the Domain Controller. The transfer of 3 MB outbound and 28 MB inbound is consistent with interactive GUI traffic (mouse/keyboard movements sent, screen frames received), confirming a working session rather than a simple probe.

Step 4: The Second-Hop Pivot to the Domain Controller: Now pivot from the first-hop workstation and look for LDAP or SMB connections into the server segment. This query does not hard-code the destination, so the second-hop target has to come from the result:

```
_path == "conn" |
id.orig_h == 10.14.10.15 |
id.resp_p in [389, 445] |
count() by id.resp_h, id.resp_p |
sort id.resp_h
```

The output:

from Investigation-Data		
1	_path == "conn"	
2	id.orig_h == 10.14.10.15	
3	id.resp_p in [389, 445]	
4	count() by id.resp_h, id.resp_p	
5	sort id.resp_h	

id	count
> {resp_h: [redacted], resp_p: 445}	1
> {resp_h: [redacted], resp_p: 389}	3

The result shows the IT admin workstation reaching 10.14.0.5 over LDAP and SMB. This is the second-hop target: the Domain Controller. The short LDAP sessions suggest Active Directory reconnaissance, and the SMB activity indicates follow-on access to the server segment.

The 13-minute gap between the RDP session start and the DC queries indicates hands-on-keyboard activity as the attacker manually navigated the session. The Incident Response (IR) scope should be to isolate both workstations immediately and treat the Domain Controller as compromised.

Answer the questions below

How many unanswered SMB SYN attempts did 10.14.22.88 generate?

23

```
_path == "conn " |
(cidr_match(10.0.0.0/8, id.orig_h) or cidr_match(172.16.0.0/12, id.orig_h)
or cidr_match(192.168.0.0/16, id.orig_h)) |
(cidr_match(10.0.0.0/8, id.resp_h) or cidr_match(172.16.0.0/12, id.resp_h)
or cidr_match(192.168.0.0/16, id.resp_h)) |
not id.orig_h == id.resp_h | count () by id.orig_h, id.resp_p | sort -r count
```

This query describes **lateral movement**, the phase in an attack where a threat actor leverages compromised credentials or exploits to move internally between devices, using administrative protocols like SMB or RDP to reach high-value assets.

id	count
> {orig_h: 10.14.22.88, resp_p: 445}	23
> {orig_h: 10.14.22.88, resp_p: 53}	9
> {orig_h: 10.14.10.15, resp_p: 389}	3
> {orig_h: 10.14.22.88, resp_p: 3389}	1
> {orig_h: 10.14.10.15, resp_p: 445}	1

What destination IP did 10.14.10.15 reach over LDAP and SMB during the second hop?

10.14.0.5

```
_path == "conn " |
id.orig_h == 10.14.10.15 | id.resp_p in [389, 445] | count() by id.resp_h, id.resp_p |
sort id.resp_h
```

This query counts administrative connection attempts (Active Directory/LDAP on port 389 and SMB on port 445) initiated by internal host `10.14.10.15`, grouped by destination IP and port to reveal potential lateral movement or internal reconnaissance.

Task 7 - Building Hunt Workflows

Queries you run but never save are lost the moment you close the tab. This task takes the ad-hoc investigation we built across this room and turns it into a named query library that any analyst on the team can pick up and run.

The Starter Hunt Library: Eight `.zed` files are pre-staged on the VM at `/home/ubuntu/artefacts/starter-queries/`. Each file contains one of the hunt queries we built across this room. Let us confirm they are all there:

```
ubuntu@tryhackme:~$ ls /home/ubuntu/artefacts/starter-queries/
beaconing-conn-highcount.zed  dns-nxdomain-volume.zed    longconn-external.zed
dns-longquery.zed             exfil-conn-asymmetric.zed  offhours-largetransfer.zed
lateralmov-smb-internal.zed   tls-badcert.zed
```

You should see 8 files. These cover the main hunt patterns from this room: beaconing, exfiltration, long connections, DNS tunnelling, bad certificates, lateral movement, and off-hours transfers.

Saving Queries in Zui: The Queries panel in Zui starts empty. We build the library by reading each query file from disk, pasting it into Zui, running it to confirm it works, and saving it. To read a query file, use `cat`:

```
ubuntu@tryhackme:~$ cat /home/ubuntu/artefacts/starter-queries/beaconing-conn-highcount.zed
_path == "conn"
| not (cidr_match(10.0.0.0/8, id.resp_h) or cidr_match(172.16.0.0/12, id.resp_h) or cidr_match(192.168.0.0/16, id.resp_h))
| count() by id.orig_h, id.resp_h, id.resp_p
| count > 50
| sort -r count
```

Copy the query text, paste it into the Zui query bar, and run it against the **Investigation-Data** pool to confirm it returns results. Once it works, save it:

1. Click the **bookmark icon** in the query bar (or press `Ctrl+S`)
2. Enter the name matching the filename, without the `.zed` extension. For example: `Beaconing-conn-highcount`

3. Click **Save** and the query appears in the Queries panel on the left

Repeat this for all 8 files. The table below shows each query name and what it hunts:

Query Name	What It Hunts
Beaconing-conn-highcount	High-frequency external connections (beacon candidates)
Exfil-conn-asymmetric	Large outbound transfers with a high send-to-receive byte ratio
LongConn-external	External sessions sustained for more than one hour
DNS-LongQuery	DNS query names longer than 60 characters (tunnel candidates)
DNS-NXDOMAIN-volume	Hosts generating high NXDOMAIN rates (DGA candidates)
TLS-BadCert	Connections with self-signed or otherwise invalid certificates
LateralMov-SMB-internal	Internal SMB connections across hosts (lateral movement candidates)
OffHours-largetransfer	Large data transfers outside business hours

Naming Convention: A consistent naming format keeps a shared library navigable. Use the pattern `[Behaviour]-[LogSource]-[KeyField]`. For example:

- Beaconing-conn-highcount
- Exfil-conn-asymmetric
- LateralMov-SMB-internal

Running the Beaconing Query with a Time Window: Once saved, click `Beaconing-conn-highcount` in the Queries panel to load it. Run the query once without a time filter so you can see the full result set and the timeline histogram above the table.

Now narrow the result set to the two-hour window between `02:00` and `04:00` UTC:

1. Move your cursor to the histogram above the results table.
2. Click and drag across the activity between `02:00` and `04:00` on 14 November 2025.
3. Release the mouse button. Zui will zoom the histogram into that selected time range and apply the time window to the active query session.
4. Run the query again if the results do not refresh automatically.

If the histogram selection is difficult to control, use an explicit timestamp filter instead. This produces the same narrowed view:

```
_path == "conn"
| ts >= 2025-11-14T02:00:00Z
| ts <= 2025-11-14T04:00:00Z
| not (cidr_match(10.0.0.0/8, id.resp_h) or cidr_match(172.16.0.0/12, id.resp_h) or cidr_match(192.168.0.0/16, id.resp_h))
| count() by id.orig_h, id.resp_h, id.resp_p
```

```
| count > 50  
| sort -r count
```

The Finance workstation and the C2 IP should still appear at the top inside the narrowed two-hour window. That is the point of this check: a real beacon keeps firing on a fixed schedule, so it remains visible in any time window that overlaps the infection period.

Documenting Null Results: Not every query returns results, and that is still useful. Let us run this query to check for Telnet activity:

```
_path == "conn" | id.resp_p == 23 | count()
```

```
from Investigation-Data
```

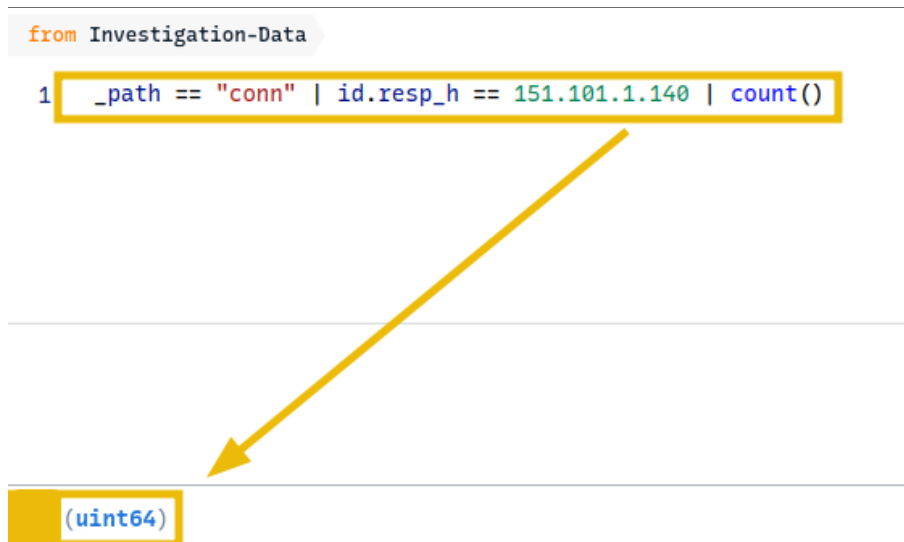
```
1 _path == "conn" | id.resp_p == 23 | count()
```



A count of zero means this dataset has been checked for Telnet and contains none. Save the query anyway and note the result. The next analyst needs to know whether something was checked and found clean, or never checked at all.

Handling False Positives: The beaconing query surfaces both the C2 IP and a Fastly CDN IP. Let us run this query to see how many connections go to the CDN address:

```
_path == "conn" | id.resp_h == 151.101.1.140 | count()
```



Once confirmed as a known CDN, add an exclusion to the beaconing query to reduce noise:

```
... | not cidr_match(151.101.0.0/16, id.resp_h) | ...
```

Note the tradeoff: an attacker routing C2 traffic through Cloudflare would disappear behind that exclusion. Document what you excluded and why, so the next person who tunes the query understands what is being traded away.



Exporting and Sharing: Export the full library via **Query > Export Queries**, which produces a JSON file containing all saved queries. Teammates import the same file via **Query > Import Queries**. A shared library means every analyst on the team is running the same queries against the same definitions, with false positives catalogued in one place.

From Ad-Hoc to Structured Hunts: A structured hunt follows a repeatable cycle:

1. **Hypothesis formulation.** "Is there beaconing activity in this environment?"
2. **Query selection.** Load `Beaconing-conn-highcount` from the library
3. **Scope adjustment.** Set a time window via the histogram and adjust thresholds for the environment's baseline
4. **Triage results.** Work through highest to lowest, classifying each as legitimate or suspicious
5. **Pivot and validate.** Follow suspicious entries into `ssl.log`, `http.log`, and `files.log`
6. **Document findings.** Record what was hunted, what was found, and what was checked but came back clean

A hunt that returns nothing is still a valid result. It means that behaviour was checked and not found in this dataset.

Answer the questions below

How many connections go to the Fastly CDN IP that appears as a false positive in the beaconing query?

55

```
_path == "conn" | id.resp_h == 151.101.1.140 | count()
```

Task 8 - Conclusion

The same Zeek data we examined in the previous room, using command-line tools, became significantly more powerful in Zui's query-first, visual investigation environment. Exploratory analysis (hunting for unknown unknowns) needs an interface that supports iteration, not one-shot retrieval. Our three hunts (beaconing, exfiltration, and lateral movement) made the case in practice.



Key Takeaways

- **Threat hunting is inductive.** Alert response starts from an indicator and works outward, while hunting starts from a hypothesis and works toward evidence. That is why we needed an iterative pivot-driven toolset rather than another command-line pipeline.
- **The three hunting postures work together.** Intel-driven hunts begin from a threat report, anomaly-driven hunts begin from a baseline, and hypothesis-driven hunts begin from a question. Real operations combine all three into a single investigation.
- **Zed turns hunting into pipeline construction.** Each pipe narrows, transforms, or aggregates, so a hunt query reads left to right as an investigative thought and each new stage is a new clause rather than a rewritten query.
- **Behavioural signatures outlast content signatures.** Hundreds of connections from one source to one destination at near-fixed intervals is a beacon regardless of payload, User-Agent, or URI. That is exactly what content-matching rules miss.
- **The saved query library is part of the SOC's organisational infrastructure.** A SOC that maintains a named, described, versioned library has documented its hunting practice. Onboarding a new analyst becomes a library review rather than months of intuition-building.

Indicators of Compromise: The investigation in this room produced a structured set of indicators:

Type	Value	Role
Source IP (compromised)	10.14.22.88 (WKST-FINANCE-04)	Beacon source, exfiltration source, lateral movement origin
Source IP (lateral hop)	10.14.10.15 (WKST-IT-ADMIN-02)	RDP target, second-hop pivot
Destination IP	194.165.16.56 (AS44477)	Cobalt Strike C2
Destination IP	185.213.154.201 (AS44477)	Staging exfiltration endpoint
Domain	update.softpatch-cdn.com	C2 SNI
Domain	backup.corpfiles-sync.com	Exfiltration host header
File	backup_archive.zip	Staged exfiltration archive (5.3 MB)
SHA256	ad73dale040200e23b2ee98e134b924f5f23e47a45207b6b304416d5aa65bdfe	Hash of backup_archive.zip
TLS fingerprint	t13d1516h2_8daaf6152771_e6bea0d95778	C2 client identification

Where the Logs End

Zui confirmed our scope cleanly: WKST-FINANCE-04 reached the IT admin workstation, which reached the Domain Controller, which gave the actor domain-level access. The TLS metadata allowed us to attribute the C2 traffic to Cobalt Strike with high confidence. What Zui cannot tell us is which commands ran in that RDP session, what was actually in the 5.3 MB archive that left at 03:15, or what the C2 channel carried in either direction. For that we have to step out of the logs and into the packets.